

domaci2_2022_0218

January 20, 2026

STUDENT: Vasilije Čarapić 2022/0218

```
[142]: import skimage as ski
import numpy as np
import matplotlib as mpl
import matplotlib.pyplot as plt
```

0.1 1. a)

1 Diskretna Furijeova transformacija 2D Gausove funkcije

Dvodimenzionalna Gausova funkcija dobija se tako što pomnožimo jednodimenzionu gausovu funkciju po kolonama:

$$g(c) = e^{-\frac{c^2}{2\sigma_{cS}^2}}$$

i jednodimenzionu gausovu funkciju po redovima:

$$g(r) = e^{-\frac{r^2}{2\sigma_{rS}^2}}$$

nakon čega dobijamo dvodimenzionalnu gausovu funkciju:

$$g(r, c) = g(r)g(c) = e^{-\frac{r^2}{2\sigma_{rS}^2} - \frac{c^2}{2\sigma_{cS}^2}}$$

gde su: - Širina i visina slike: (NW) i (NH)

- Broj tačaka DFT po horizontali i vertikalni: (NW) i (NH)

1.1 2. Aproksimacija integralom

Pošto Gauss funkcija brzo opada, sume od 0 do N-1 možemo aproksimirati kao sume od $-\infty$ do $+\infty$, odakle se dobija:

$$G_r(k) = \sum_{r=-\infty}^{+\infty} g_r(r) e^{-j2\pi \frac{k}{N_H} r}$$

$$G_c(l) = \sum_{c=-\infty}^{+\infty} g_c(c) e^{-j2\pi \frac{k}{N_W} l}$$

A nakon toga možemo tu sumu aproksimirati integralom koji ako malo bolje pogledamo jeste Furijeova transformacija, pa rešavanjem integrala dobijemo sledeći rezultat:

1.2 Gr(k) = FT{g(r)}:

$$G_r(k) \approx \int_{-\infty}^{\infty} e^{-r^2/(2\sigma_{rS}^2)} e^{-j2\pi \frac{k}{N_H} r} dr = \sigma_{rS} \sqrt{2\pi} e^{-2\pi^2 \sigma_{rS}^2 \frac{k^2}{N_H^2}}$$

1.3 Gc(l) = FT{g(c)}:

$$G_c(l) \approx \int_{-\infty}^{\infty} e^{-c^2/(2\sigma_{cS}^2)} e^{-j2\pi \frac{l}{N_W} c} dc = \sigma_{cS} \sqrt{2\pi} e^{-2\pi^2 \sigma_{cS}^2 \frac{l^2}{N_W^2}}$$

Čime se dobijaju furijeove transformacije kao. Gledajući definiciju

1.4 3. 2D DFT

2D diskretna Furijeova transformacija se dobija kao proizvod 1D transformacija po redovima i kolonama:

$$G_F(k, l) = G_r(k) \cdot G_c(l) = C e^{-2\pi^2 \sigma_{rS}^2 \frac{k^2}{N_H^2} - 2\pi^2 \sigma_{cS}^2 \frac{l^2}{N_W^2}}$$

gde je:

$$C = \sigma_{rS} \sigma_{cS} 2\pi$$

konstanta normalizacije.

1.5 4. Standardne devijacije u frekvencijskom domenu

- 2D DFT Gaussove funkcije je **takođe Gausova** sto se lako da proveriti jer obe imaju isti oblik Gausove funkcije.
- Ako uporedimo sa recimo ovom funkcijom u prostornom domenu

$$g(r, c) = g(r)g(c) = e^{-\frac{r^2}{2\sigma_{rS}^2} - \frac{c^2}{2\sigma_{cS}^2}}$$

mozemo lako odrediti parametar

$$\sigma_{rS}$$

i parametar

$$\sigma_{cS}$$

- Standardne devijacije u frekvencijskom domenu zavise od standardnih devijacija u prostornom domenu i broja tačaka diskretizacije:

$$\sigma_{rF}^2 = f(\sigma_{rS}, N_H) = \frac{N_H^2}{4 * \pi^2 * \sigma_{rS}^2}, \quad \sigma_{cF}^2 = f(\sigma_{cS}, N_W) = \frac{N_W^2}{4 * \pi^2 * \sigma_{cS}^2}$$

Diskretna DFT Gauss funkcija je **takođe Gaussova**, sa standardnim devijacijama:

$$\sigma_r^F = \frac{N_H}{2\pi\sigma_{rS}}, \quad \sigma_c^F = \frac{N_W}{2\pi\sigma_{cS}}$$

- Odnos standardnih devijacija u prostornom i frekvencijskom domenu:

$$\sigma_F \sim \frac{N}{2\pi\sigma^S}$$

1.6 1. b)

```
[ ]: def lpfilter(filt_type, Ny, Nx, sigma, n=1):

    if (Ny%2 == 0):
        y = np.arange(0,Ny) - Ny/2 + 0.5
    else:
        y = np.arange(0,Ny) - (Ny-1)/2

    if (Nx%2 == 0):
        x = np.arange(0,Nx) - Nx/2 + 0.5
    else:
        x = np.arange(0,Nx) - (Nx-1)/2

    X, Y = np.meshgrid(x, y)

    D = np.sqrt(np.square(X) + np.square(Y))

    if filt_type == 'gaussian':
        filter_mask = np.exp(-np.square(D)/(2*np.square(sigma)))
    elif filt_type == 'btw':
        filter_mask = 1/(1+(D/sigma)**(2*n))
    elif filt_type == 'ideal':
        filter_mask = np.ones([Ny, Nx])
        filter_mask[D>sigma] = 0
    else:
        print('Greška! Nije podržan tip filtra: ', filt_type)
        return

    return filter_mask
```

```
[ ]: img_path = "../sekvence/"
```

Funkcija `filter_image` primenjuje **niskopropusno filtriranje slike u frekvencijskom domenu**.

Ulazna slika se najpre transformiše u frekvencijski domen pomoću dvodimenzionalne Furijeove transformacije (FFT), čime se dobija njena spektralna reprezentacija.

Zatim se generiše Gaussian niskopropusni filter, koji potiskuje visoke frekvencije i omogućava zaglađivanje slike.

Filtriranje se ostvaruje množenjem spektra slike i frekvencijskog filtra.

Nakon filtriranja, slika se vraća u prostorni domen primenom inverzne Furijeove transformacije (IFFT).

Na kraju se vrednosti piksela ograničavaju na opseg $[0, 1]$, čime se obezbeđuje ispravan prikaz filtrirane slike.

```
[ ]: def filter_image(img_org, sigma):
    [Ny, Nx] = np.shape(img_org)

    img = img_org.copy()
    #slika u frekv domenu
    img_fft = np.fft.fftshift(np.fft.fft2(img))

    #filtrar
    filt_freq = lpfilter("gaussian", Ny, Nx, sigma)

    #filtrirana slika

    img_fft_filt = img_fft * filt_freq

    # vrati nazad u realni domen
    img_filt = np.real(np.fft.ifft2(np.fft.ifftshift(img_fft_filt)))
    img_filt = np.clip(img_filt, 0, 1)

    return img_filt
```

```
[ ]: def plot_img(img, filtered_img):
    '''prikazuje slika, filtrirana slika na jednom grafiku '''
    fig, axes = plt.subplots(1, 2, figsize = (10, 6), dpi = 100)
    ax = axes.ravel()

    ax[0].imshow(img, cmap = "gray"); ax[0].set_axis_off(); ax[0].
    ↪set_title("Ulazna slika");
    ax[1].imshow(filtered_img, cmap = "gray"); ax[1].set_axis_off(); ax[1].
    ↪set_title("Filtrirana slika");

    plt.tight_layout()
    plt.show()
```

```
[ ]: img = ski.util.img_as_float(ski.io.imread(img_path + "square_256x256.tif"))
img_fft_5 = filter_image(img, 5)
img_fft_10 = filter_image(img, 10)
img_fft_20 = filter_image(img, 20)

plot_img(img, img_fft_5)
plot_img(img, img_fft_10)
plot_img(img, img_fft_20)
```

2 Filtriranje slike u frekvencijskom i prostornom domenu

Zadata je slika `square_256x256.png`. Potrebno je izvršiti filtriranje u frekvencijskom domenu koristeći Gausov **LP filter** sa standardnim devijacijama:

$$\sigma_F = 5, \quad 10, \quad 20$$

2.1 Određivanje ekvivalentnih parametara prostorne Gausove funkcije

Ekvivalentna standardna devijacija u prostornom domenu se dobija iz prethodno izvedene relacije:

$$\sigma_S = \frac{N}{2\pi\sigma_F}$$

Gde je ($N = 256$) (veličina slike po x i y).

Za zadate vrednosti koje imamo:

$$\sigma_F = 5, \quad 10, \quad 20$$

dobijamo vrednosti:

$$\sigma_S = 8.15, \quad 4.07, \quad 2.04$$

Napomena: **S je isti za x i y** jer je filter simetričan.

2.2 Prostorna Gausova funkcija

Prostorni ekvivalent filtra se definiše kao:

$$G(x, y) = e^{-\frac{x^2+y^2}{2\sigma_S^2}}$$

Filtriranje slike u prostornom domenu se vrši konvolucijom sa ovim Gausovim filterom.

```
[ ]: img = ski.util.img_as_float(ski.io.imread(img_path + "square_256x256.tif"))

img_gauss_5 = ski.filters.gaussian(img, sigma = 8.15)
img_gauss_10 = ski.filters.gaussian(img, sigma = 4.07)
img_gauss_20 = ski.filters.gaussian(img, sigma = 2.04)

plot_img(img, img_gauss_5)
plot_img(img, img_gauss_10)
plot_img(img, img_gauss_20)
```

2.3 Upoređivanje rezultata

Razlika između filtriranja u prostornom i frekvencijskom domenu se kvantifikuje pomoću najveće apsolutne greške:

$$\text{maxAbsErr} = \max |Im_{\text{freq}} - Im_{\text{space}}|$$

Očekivano, razlika je **numerička greška** zbog aproksimacije i diskretizacije DFT.

```
[ ]: max_abs_err1 = abs(np.max(img_fft_5 - img_gauss_5))
max_abs_err2 = abs(np.max(img_fft_10 - img_gauss_10))
max_abs_err3 = abs(np.max(img_fft_20 - img_gauss_20))

print("max abs err 1:", max_abs_err1)
print("max abs err 2:", max_abs_err2)
print("max abs err 3:", max_abs_err3)
```

```
[ ]: img = ski.util.img_as_float(ski.io.imread(img_path + "square_256x1024.tif"))
img_sigma_5 = filter_image(img, 5)
img_sigma_10 = filter_image(img, 10)
img_sigma_20 = filter_image(img, 20)

plot_img(img, img_sigma_5)
plot_img(img, img_sigma_10)
plot_img(img, img_sigma_20)
```

2.4 Filtriranje slike u frekvencijskom domenu

Da bi rezultat filtriranja u prostornom domenu imao jednak stepen zamućenja po redovima i kolonama, standardne devijacije Gaussovog filtra u frekvencijskom domenu moraju biti proporcionalne dimenzijama slike, tj. moraju ispunjavati uslov:

$$\frac{F_x}{N_x} = \frac{F_y}{N_y}$$

gde su N_x i N_y broj kolona i redova slike, respektivno. Za konkretnu sliku dimenzija 256×1024 važi odnos:

$$F_x : F_y = 1024 : 256 = 4 : 1$$

odnosno standardna devijacija filtra po horizontalnoj osi mora biti četiri puta veća nego po vertikalnoj osi kako bi se postigao jednak stepen zamućenja u oba pravca.

2.5 1. c)

Funkcija `lpfilter_anisotrop` generiše **anizotropni niskopropusni filter u frekvencijskom domenu**.

Filter se formira za zadate dimenzije slike (N_y , N_x) i različite standardne devijacije po x i y osi (`sigma_x`, `sigma_y`), čime se omogućava anizotropno filtriranje.

Najpre se formiraju koordinate u frekvencijskom domenu, pri čemu se vodi računa o parnim i neparnim dimenzijama slike kako bi se frekvencijska osa pravilno centrirala.

Zatim se izračunava normalizovano rastojanje D , koje uzima u obzir različite vrednosti `sigma_x` i `sigma_y`.

U zavisnosti od parametra `filt_type`, funkcija može generisati Gaussian, Butterworth (`btw`) ili idealni niskopropusni filter.

Na kraju se vraća maska filtra koja se koristi za frekvencijsko filtriranje slike.

```
[ ]: def lpfilter_anisotrop(filt_type, Ny, Nx, sigma_x, sigma_y, n = 1):
    if(Ny % 2 == 0):
        # 0 1 2 3 => (-2 -1 0 1) + 0.5 => (-1.5 -0.5 0.5 1.5)
        y = np.arange(0, Ny) - (Ny / 2) + 0.5
    else:
        y = np.arange(0, Ny) - (Ny - 1) / 2

    if(Nx % 2 == 0):
        x = np.arange(0, Nx) - (Nx / 2) + 0.5
    else:
        x = np.arange(0, Nx) - (Nx - 1) / 2

    X, Y = np.meshgrid(x, y)

    #izracunam D kao
    D = np.sqrt(np.square(X / sigma_x) + np.square(Y / sigma_y))

    if filt_type == "gaussian":
        filter_mask = np.exp(-X**2 / (2 * sigma_x ** 2) - Y ** 2 / (2 * sigma_y
→** 2))
    elif filt_type == "btw":
        filter_mask = 1 / (1 + D **(2 * n))
    elif filt_type == "ideal":
        filter_mask = np.ones([Ny, Nx])
```

```

        filter_mask[D > 1] = 0
    else:
        print("Error! That filter type is not available", filt_type)
        return
    return filter_mask

```

S obzirom na to da se zahteva da parametri filtra u frekvencijskom domenu budu ekvivalentni parametrima u prostornom domenu, kao i da je filter **izotropan** ($\sigma_{s_x} = \sigma_{s_y} = \sigma_s$), moguće je odrediti odgovarajuće parametre u frekvencijskom domenu.

Standardne devijacije u frekvencijskom domenu računaju se na sledeći način:

$$\sigma_{f_x} = \frac{N_x}{2\pi\sigma_s}$$

$$\sigma_{f_y} = \frac{N_y}{2\pi\sigma_s}$$

gde su N_x i N_y dimenzije slike u x i y pravcu, dok σ_s predstavlja standardnu devijaciju filtra u prostornom domenu.

```

[ ]: def calc_freq_sigma(sigma_s, img):
    Ny, Nx = img.shape
    #izotropan filter sigma_s = sigma_s_x = sigma_s_y
    sigma_x_f, sigma_y_f = Nx / (2 * np.pi * sigma_s), Ny / (2 * np.pi *
    ↪sigma_s)
    return sigma_x_f, sigma_y_f

```

```

[ ]: def filter_c(img_org, sigma_x, sigma_y):
    """funkcija koja filtrira sliku
    1. prebaci sliku u fft domen
    2. pomnozi sliku sa filtrom u fft domenu
    3. vrati u prostorni domen sliku
    4. iseci na (0, 1) opseg ako je neka vrednost izvan opsega
    """

    [Ny, Nx] = np.shape(img_org)

    img = img_org.copy()
    # slika u frekv domenu
    img_fft = np.fft.fftshift(np.fft.fft2(img))

    # filter
    filt_freq = lpfilter_anisotrop("gaussian", Ny, Nx, sigma_x, sigma_y)

    # filtrirana slika

    img_fft_filt = img_fft * filt_freq

```

```

# vrati nazad u realni domen
img_filt = np.real(np.fft.ifft2(np.fft.ifftshift(img_fft_filt)))
img_filt = np.clip(img_filt, 0, 1)

return img_filt

```

Funkcija `filter_and_plt_img` primenjuje frekvencijsko filtriranje slike za **više vrednosti parametra** `sigma_s`.

Za svaku vrednost `sigma_s` izračunavaju se odgovarajući parametri filtra u frekvencijskom domenu (`sigma_x_f`, `sigma_y_f`).

Zatim se slika filtrira pomoću funkcije `filter_c`, a rezultati se prikazuju korišćenjem funkcije `plot_img`.

Na ovaj način omogućeno je poređenje uticaja različitih vrednosti `sigma_s` na rezultat filtriranja.

```

[ ]: def filter_and_plt_img(img, sigma_s):
    sigma_x_f_256, sigma_y_f_256 = calc_freq_sigma(sigma_s, img)
    for sigma_x, sigma_y in zip(sigma_x_f_256, sigma_y_f_256):
        filtered_img = filter_c(img, sigma_x, sigma_y)
        plot_img(img, filtered_img)

[ ]: sigma_s = np.array([5, 10, 20])
img_256_square = ski.util.img_as_float(ski.io.imread(img_path + "square_256x256.
    ↳tif"))
img_1024_square = ski.util.img_as_float(ski.io.imread(img_path + "
    ↳square_256x1024.tif"))
filter_and_plt_img(img_256_square, sigma_s)
filter_and_plt_img(img_1024_square, sigma_s)

```

Prve dve funkcije, `mean_value_spatial` i `mean_value_freq`, služe za merenje **prosečnog vremena izvršavanja filtriranja** u prostornom, odnosno frekvencijskom domenu.

Za zadati broj ponavljanja (`num_of_times`), svaka funkcija više puta izvršava odgovarajući filter, meri vreme trajanja svakog izvršavanja i na kraju vraća srednju vrednost vremena.

Funkcija `spatial_filtering` koristi `mean_value_spatial` kako bi za niz vrednosti `sigma` izračunala prosečna vremena filtriranja u prostornom domenu.

Za svaku vrednost parametra `sigma` rezultat se smešta u niz `times`, koji predstavlja zavisnost vremena izvršavanja od širine filtra.

Analogno tome, funkcija `freq_filtering` računa prosečna vremena filtriranja u frekvencijskom domenu.

Za svaku vrednost `sigma` najpre se izračunavaju odgovarajući parametri filtra u frekvencijskom domenu (`sigma_x` i `sigma_y`), nakon čega se poziva funkcija `mean_value_freq`.

Na ovaj način omogućeno je direktno poređenje brzine filtriranja u prostornom i frekvencijskom domenu za različite vrednosti parametra `sigma`.

```

[ ]: def mean_value_spatial(img, sigma, num_of_times):
    mean_values = []
    for i in range(0, num_of_times):

```

```

        start = time.time()
        ski.filters.gaussian(img, sigma = sigma)
        end = time.time()
        interval = end - start
        mean_values.append(interval)
    return np.mean(mean_values)

```

```

[ ]: def mean_value_freq(img, sigma, num_of_times, sigma_x, sigma_y):
    mean_values = []
    for i in range(0, num_of_times):
        start = time.time()
        filter_c(img, sigma_x, sigma_y)
        end = time.time()
        interval = end - start
        mean_values.append(interval)
    return np.mean(mean_values)

```

```

[ ]: def spatial_filtering(img, sigmas, num_of_times):
    times = np.zeros(len(sigmas))
    for i, sigma in enumerate(sigmas):
        times[i] = mean_value_spatial(img, sigma, num_of_times)
    return times

```

```

[ ]: def freq_filtering(img, sigmas, num_of_times):
    times = np.zeros(len(sigmas))
    Ny, Nx = img.shape
    for i, sigma in enumerate(sigmas):
        sigma_x = Nx / (2 * np.pi * sigma)
        sigma_y = Ny / (2 * np.pi * sigma)
        times[i] = mean_value_freq(img, sigma, num_of_times, sigma_x, sigma_y)
    return times

```

```

[ ]: import time

```

U ovoj ćeliji se učitavaju dve test slike i konvertuju u format sa vrednostima u opsegu ([0, 1]). Zatim se definiše niz vrednosti parametra **sigma**, koji obuhvata opseg od 1 do 200.

Nakon toga se meri prosečno vreme izvršavanja filtriranja u **prostornom domenu** pomoću funkcije **spatial_filtering**.

Paralelno se meri i vreme izvršavanja filtriranja u **frekvencijskom domenu** korišćenjem funkcije **freq_filtering**, što omogućava poređenje performansi oba pristupa.

```

[ ]: img_256_square = ski.util.img_as_float(ski.io.imread(img_path + "square_256x256.
    ↪tif"))
img_1024_square = ski.util.img_as_float(ski.io.imread(img_path + "
    ↪square_256x1024.tif"))

```

```

sigmas = np.array(np.arange(1, 201, 1))
spatial_times_256 = spatial_filtering(img_256_square, sigmas, 3)
freq_times_256 = freq_filtering(img_256_square, sigmas, 3)

```

3 Opis crtanja grafika

Ova ćelija crta grafik sa dva niza podataka:

- **Vremena izvršavanja prostornog filtriranja** (spatial_times_256) prikazana crvenom linijom.
- **Vremena izvršavanja frekvencijskog filtriranja** (freq_times_256) prikazana plavom linijom.

X-osa predstavlja vrednosti parametra σ_s , dok y-osa prikazuje vreme izvršavanja u sekundama.

Cilj ćelije je prikazati oba skupa podataka na jednom grafikonu radi vizualnog poređenja vremena izvršavanja za različite vrednosti σ .

```

[ ]: plt.figure(figsize=(12, 7))

plt.plot(sigmas, spatial_times_256, 'r-', linewidth=3, label='Prostorni domen')
plt.plot(sigmas, freq_times_256, 'b-', linewidth=3, label='Frekvencijski domen')

plt.xlabel(r"$\sigma_s$")
plt.ylabel('Vreme izvršavanja (s)', fontsize=12)
plt.title('Oba dijagrama na jednom grafiku', fontsize=14, fontweight='bold')
plt.legend(fontsize=12)
plt.grid(True, alpha=0.3)
plt.xlim([1, 200])

plt.tight_layout()
plt.show()

```

4 Analiza rezultata grafika izvršavanja filtra

Na grafiku su prikazana vremena izvršavanja dve vrste filtriranja slike: prostorno (crvena linija) i frekvencijski domen (plava linija) u zavisnosti od parametara (σ_s, σ_f).

- Vreme izvršavanja prostornog filtriranja značajno raste sa povećanjem vrednosti σ , što je očekivano jer veći parametar znači veći prozor i samim tim više operacija po pikselu.
- Vreme frekvencijskog filtriranja je gotovo konstantno i vrlo nisko, što pokazuje da je frekvencijski pristup mnogo efikasniji, naročito za veće vrednosti σ .
- Male oscilacije kod frekvencijskog domena mogu biti posledica varijacija u računarskim resursima ili implementacionim detaljima, ali su zanemarljive u poređenju sa rastom kod prostornog filtriranja.
- Rezultati jasno pokazuju prednost frekvencijskog filtriranja kod obrade slika sa velikim vrednostima parametra σ .

4.1 2) Deo

```
[ ]: import numpy as np
import matplotlib.pyplot as plt
import matplotlib as mpl
import skimage as ski
from ipywidgets import widgets
```

```
[ ]: data_dir = "../sekvence/"
```

Pošto je slika data u **sRGB** domenu, prvi korak koji je potrebno obaviti jeste njena konverzija iz sRGB domena u **linearni domen**.

Ako je gamma vrednost definisana kao

$$\gamma = \frac{1}{2.2}$$

tada bi se pri prelasku iz linearnog domena u sRGB domen svaka vrednost stepenovala sa

$$\frac{1}{2.2}.$$

Iz tog razloga, obrnuti proces — prelazak iz sRGB u linearni domen — podrazumeva stepenovanje sa 2.2.

Zbog toga je uvedena funkcija **srgb_to_linear**, koja omogućava pravilnu transformaciju vrednosti piksela u linearni prostor.

Na kraju obrade, neophodno je sliku iz linearnog domena ponovo vratiti u **sRGB domen**, što se postiže primenom funkcije **linear_to_srgb**.

```
[ ]: def srgb_to_linear(img):
    img = np.clip(img, 0, 1)
    linear_img = img ** 2.2
    return linear_img
```

```
[ ]: def linear_to_srgb(img):
    img = np.clip(img, 0, 1)
    srgb_img = img ** (1 / 2.2)
    return srgb_img
```

Funkcija **deconvolute_channel** vrši dekonvoluciju jednog kanala slike u frekvencijskom domenu. Ulazna slika se najpre transformiše u frekvencijski domen pomoću 2D FFT-a. Kernel zamućenja se zatim proširuje nulama kako bi imao iste dimenzije kao slika i takođe se transformiše u frekvencijski domen.

Dekonvolucija se realizuje deljenjem spektra zamućene slike sa spektrom kernela, uz primenu regularizacionog faktora koji smanjuje uticaj šuma i stabilizuje proces. Nakon toga se rezultat vraća u prostorni domen pomoću inverznog FFT-a, uzimajući realni deo signala.

Na kraju, vrednosti piksela se ograničavaju na opseg $[0, 1]$ i izlazna slika se vraća na originalne dimenzije.

```
[ ]: def deconvolute_channel(img, kernel):
    orig_height, orig_width = img.shape
    img_fft = np.fft.fftshift(np.fft.fft2(img))
    k = 1e-5

    kernel_height, kernel_width = kernel.shape
    H, W = img.shape[:2]
    kernel_padded = np.zeros((H, W))
    kernel_padded[:kernel_height, :kernel_width] = kernel

    kernel_fft = np.fft.fftshift(np.fft.fft2(kernel_padded))

    W = (abs(kernel_fft) ** 2) / (abs(kernel_fft) ** 2 + k)

    # zamucena slika / faktor zamucenja
    img_fft_est = (img_fft / kernel_fft) * W

    img_est = np.real(np.fft.ifft2(np.fft.ifftshift(img_fft_est)))
    img_est = np.clip(img_est, 0, 1)

    return img_est[:orig_height, :orig_width]
```

funkcija deconvolute samo crta dve slike i prikazuje ih.

```
[ ]: def deconvolute(img, kernel):
    if img.ndim != 3: return;
    output_img = np.zeros_like(img)
    for channel in range(img.shape[2]):
        output_img[:, :, channel] = deconvolute_channel(img[:, :, channel], ↵
↵kernel)
    return output_img
```

funkcija plt_img radi dekonvoluciju RGB slike po svakom kanalu.

```
[ ]: def plt_img(img1, img2, title2):
    fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(12,8), dpi=90)
    ax = axes.ravel()

    ax[0].imshow(img1); ax[0].set_title('Ulazna slika')
    ax[1].imshow(abs(img2), cmap='gray'); ax[1].set_title(title2);
    plt.show()
```

```
[ ]: def vertical_notch_mask_mesh(Ny, Nx, notch_width=3, dc_radius=8):
    """
    Vertikalni notch filter koristeći meshgrid koordinatni sistem.

    Ny, Nx      : dimenzije FFT-a (height, width)
```

```

notch_width : širina vertikalne linije
dc_radius   : koliko piksela oko centra ne diramo
"""
# horizontalne i vertikalne koordinate centrirane
if Ny % 2 == 0:
    y = np.arange(0, Ny) - Ny/2 + 0.5
else:
    y = np.arange(0, Ny) - (Ny-1)/2

if Nx % 2 == 0:
    x = np.arange(0, Nx) - Nx/2 + 0.5
else:
    x = np.arange(0, Nx) - (Nx-1)/2

X, Y = np.meshgrid(x, y)
D = np.sqrt(X**2 + Y**2)

# vertikalni notch (Gaussian po X)
mask = 1 - np.exp(-(X**2) / (2 * notch_width**2))

# sačuvaj DC oko centra
mask[D < dc_radius] = 1.0

return mask

```

```

[ ]: def apply_vertical_notch_filter(img, notch_width = 3, dc_radius=10):
    """
    Primeni vertikalni notch filter na linearni (FFT) kanal slike.

    image_linear : slika u linearnom domenu (H,W,3)

    return: vrati mi sliku filtriranu notch filtrom
    """
    filtered_image = np.zeros_like(img)
    for channel_index in range(img.shape[2]):
        # FFT kanala
        channel_fft = np.fft.fftshift(np.fft.fft2(img[:, :, channel_index]))

        Ny, Nx = channel_fft.shape
        # napravim masku
        notch_mask = vertical_notch_mask_mesh(Ny, Nx, notch_width, dc_radius)

        # filtriram
        filtered_fft = channel_fft * notch_mask

        # inversna FFT

```

```

        filtered_image[:, :, channel_index] = np.real(np.fft.ifft2(np.fft.
↪ifftshift(filtered_fft)))

    return np.clip(filtered_image, 0, 1)

```

funkcija `show_img_fft` prikazuje spektar slika u određenim trenucima.

Ovo nam pomaže da odredimo u kojim delovima slike se nalazi šum i koju vrstu filtra da primenimo da bi se otarasili šuma.

Da bi se bolje videlo prvo prebacimo **RGB->Grayscale** nakon čega prebacimo sliku u frekvencijski domen odakle jasno vidimo koje linije dalje ukloniti.

```

[ ]: def show_img_fft(img, text):
    img_gray = ski.color.rgb2gray(img)
    img_fft = np.fft.fftshift(np.fft.fft2(img_gray))
    fig, ax = plt.subplots(nrows = 1, ncols = 1, dpi = 80, figsize = (10, 8))
    ax.set_title(text)
    ax.imshow(np.log(1 + abs(img_fft)), cmap = 'gray')
    plt.show()

```

4.1.1 Proces obrade slike

Proces obrade započinje učitavanjem ulazne slike, koja je inicijalno predstavljena u **sRGB domenu**. S obzirom na to da sRGB prostor nije linearan, prvi korak obrade podrazumeva konverziju slike iz sRGB domena u **linearni domen**. Ovaj korak je neophodan kako bi sve naredne operacije bile matematički ispravne i fizički smislenije.

Mozemo primetiti da kada se nacrtaju spektar linearne slike, postoji najveća smetnja vertikalno u spektru.

Iz upravo ovog razloga pored klasičnog odzamućenja slike napravljen je notch filtar koji će se otarasiti tih smetnji.

Nakon prelaska u linearni domen, nad slikom se vrši **dekonvolucija**, čiji je cilj uklanjanje ili ublažavanje efekata zamućenja. Dekonvolucija se sprovodi u frekvencijskom domenu, korišćenjem poznatog kernela zamućenja, uz odgovarajuću regularizaciju radi smanjenja uticaja šuma, Filtar koji koristimo je Wienerov filtar koji će lepo isfiltrirati šum na slici.

Dobijena slika se zatim dodatno obrađuje primenom **notch filtra**, koji se koristi za potiskivanje neželjenih frekvencijskih komponenti, najčešće periodičnog šuma ili interferencija prisutnih u slici. Mogao je biti i isfiltriran i deo koji ima horizontalnu liniju u spektru ali ako bismo njega isfiltrirali i bitne komponente sa slike bi nam bile upropaštene.

Nakon završetka svih operacija u linearnom domenu, slika se ponovo konvertuje iz linearnog u **sRGB (RGB) domen**, kako bi bila pogodna za prikaz na standardnim uređajima. Time se završava kompletan proces obrade slike.

```

[ ]: img = ski.util.img_as_float(ski.io.imread(data_dir + "etf_blur.png"))

    kernel = np.loadtxt(data_dir + "blur_kernel.txt")
    kernel_fft = np.fft.fftshift(np.fft.fft2(kernel))

```

```

img_linear = srgb_to_linear(img)
show_img_fft(img_linear, "Spektar linearne slike")

deblurred_img = deconvolute(img_linear, kernel)
show_img_fft(deblurred_img, "Spektar odzamućene slike")

result_img = apply_vertical_notch_filter(deblurred_img)
show_img_fft(result_img, "Spektar slike nakon notch filtra")

srgb_img = linear_to_srgb(result_img)
show_img_fft(srgb_img, "Spektar srgb slike nazad")

```

```

[ ]: plt_img(img, kernel, "Kernel:")
plt_img(img, kernel_fft, "FFT Kernel:")
plt_img(img, srgb_img, "restaurirana slika:")

```

```

[ ]: def show_img(srgb_img, notch_width, dc_radius):

    result_img = apply_vertical_notch_filter(srgb_img, notch_width, dc_radius)
    np.clip(result_img, 0, 1)
    fig, ax = plt.subplots(figsize = (12,8), dpi = 100)
    ax.imshow(result_img)

```

Da bi odredili pogodnu sirinu potrebnu za koriscenje notch_filtra koriscen je **widgets.interact**. Parametar notch_width je sirina u spektru koju obuhvata filtar a dc_radius je velicina DC komponente od centra koju zelimo saćuvati.

Na kraju je izabrano da parametri budu **dc_radius = 31** i **notch_width = 3**.

```

[ ]: #widgets.interact(show_img, srgb_img = widgets.fixed(srgb_img), notch_width =
    ↪ (1,100), dc_radius = (0, 100))

```

```

[ ]: show_img(srgb_img, 3, 31)

```

4.2 DEO 3)

```

[ ]: data_dir = "../sekvence/"
import scipy.ndimage as ndi

```

Kao što je opisano u poglavlju, prvo što se treba uraditi jeste napisati funkciju **estimate_noise_var**. Implementirane su tri funkcije: **estimate_noise_var** kao glavna funkcija, **calc_var** kao pomoćna funkcija za računanje varijanse i **estimate_win** za određivanje odgovarajuće veličine prozora.

Funkcija **calc_var** služi za računanje varijanse signala. Lokalna varijansa se može izračunati pomoću sledeće formule:

$$\text{Var}(X) = \mathbb{E}(X^2) - (\mathbb{E}(X))^2$$

gde je $\mathbb{E}(X)$ srednja vrednost signala, dok $\mathbb{E}(X^2)$ predstavlja srednju vrednost kvadrata signala unutar posmatranog prozora. Ovaj izraz predstavlja prvi deo implementacije funkcije.

Drugi deo funkcije predstavlja procenu varijanse pomoću histograma. Najpre se izračunava histogram vrednosti signala, zatim se određuju centri histogram-binova. Kao reprezentativna vrednost uzima se maksimalna vrednost histograma, odnosno vrednost u centru histogram-bina sa najvećom učestalošću. Na osnovu ovih koraka dobija se procena varijanse šuma, koja se na kraju vraća kao izlaz funkcije.

```
[ ]: def calc_var(noise_img, win):
    '''
    :param noise_img: slika sa sumom
    :param win: velicina prozora
    :return: varijansa = srednja vrednost bina sa maksimalnom vrednoscu onog
    ↪ bina koji ima histogram
    binovi_centri = [1.5, 2.5, 3.5, 4.5], histogram_vrednosti = [1,2,3,4]
    ↪ varijansa = 4
    '''
    local_mean = ndi.uniform_filter(noise_img, size=win)
    local_sqr_mean = ndi.uniform_filter(noise_img ** 2, size=win)

    local_var = local_sqr_mean - local_mean ** 2

    # vrednost varijanse prethodne
    hist, bin_edges = np.histogram(local_var, bins=256)

    # indeks maks bina prethodne
    bin_centers = (bin_edges[:-1] + bin_edges[1:]) / 2

    var = bin_centers[np.argmax(hist)]

    return var
```

Funkcija `estimate_win` računa varijansu za prethodnu veličinu prozora i varijansu za trenutnu veličinu prozora. Računanje varijansi, a samim tim i određivanje veličine prozora koji zadovoljava uslove, prestaje kada je ispunjena sledeća nejednakost:

$$\text{Var}(\text{currWin}) - \text{Var}(\text{prevWin}) < \text{tol}$$

Ukoliko ova nejednakost ne može biti ispunjena, a dostigne se maksimalna dozvoljena veličina prozora, kao konačno rešenje se uzima maksimalna veličina prozora, a odgovarajuća varijansa se prihvata kao rezultat.

```
[ ]: def estimate_win(noise_img, min_win, max_win, curr_win, prev_win,
    tol = 0.001):
```

```

#varijansa prethodne
var_prev = calc_var(noise_img, prev_win)

while curr_win <= max_win:

    var_curr = calc_var(noise_img, curr_win)

    if abs(var_curr - var_prev) < tol:
        return curr_win

    var_prev = var_curr
    curr_win += 2

return max_win

```

Funkcija `estimate_noise_var` proverava vrednost parametra `win`. Ukoliko je veličina prozora jednaka nuli, funkcija poziva `estimate_win`, koja automatski određuje odgovarajuću veličinu prozora na osnovu promene varijanse.

U slučaju da je zadati parametar `win` različit od nule, ta vrednost se direktno koristi kao veličina prozora za dalje računanje.

Nakon određivanja veličine prozora, funkcija računa varijansu šuma za dati prozor korišćenjem prethodno definisane funkcije za računanje varijanse.

Originalno, funkcija `estimate_noise_var` je bila predviđena da vraća samo procenjenu varijansu šuma. Međutim, zbog zahteva iz tačaka b) i c), funkcija je izmenjena tako da kao izlaz vraća i procenjenu varijansu i odgovarajuću veličinu prozora.

```

[ ]: def estimate_noise_var(noise_img, win = 0, tol = 0.001):
    min_win, max_win = 3, 51

    if win == 0:
        prev_win, curr_win = min_win, min_win + 2
        win = estimate_win(noise_img, min_win, max_win, curr_win, prev_win)

    var = calc_var(noise_img, win)
    return var, win

```

```

[ ]: img = ski.io.imread(data_dir + "lena.tif")
    var, win = estimate_noise_var(img, 0, 1)
    print("var = {0:2f}".format(var))

```

4.3 b)

```

[ ]: data_dir = "../sekvenca/"
    lena_img = ski.util.img_as_float(ski.io.imread(data_dir + "lena.tif"))

```

```
[ ]: def local_hist(noise_img, curr_win):
    '''
    vraca maks vrednost histograma varijansi
    '''
    kernel = np.ones((curr_win, curr_win)) / (curr_win ** 2)

    local_mean = ndi.uniform_filter(noise_img, size=curr_win)
    local_sqr_mean = ndi.uniform_filter(noise_img ** 2, size=curr_win)

    local_var = local_sqr_mean - local_mean ** 2

    # histogram varijanse
    hist, bin_edges = np.histogram(local_var, bins = 64)

    hist_max = np.max(hist)

    return hist_max
```

Funkcija `get_result` vrši pet (jer se tražilo od 5 do 10 merenja) uzastopnih merenja varijanse i maksimalnih vrednosti histograma za svaku veličinu prozora. U svakoj od pet iteracija formira se lista koja sadrži varijanse za sve moguće veličine prozora, počev od minimalne do maksimalne vrednosti prozora.

Nakon što se izvrši svih pet merenja, za svaku veličinu prozora posebno se analiziraju dobijeni rezultati. Naime, za dati prozor (npr. prozor veličine 3), uzimaju se sve izmerene varijanse iz pet iteracija koje pripadaju datom prozoru sa veličinom 3 i računa se njihova srednja vrednost, čime se dobija srednja varijansa za tu veličinu prozora. Nad istim skupom vrednosti zatim se računa standardna devijacija. Na analogan način obrađuju se i maksimalne vrednosti histograma za ovu veličinu prozora.

Ovaj postupak se ponavlja za svaku moguću veličinu prozora.

Na kraju, funkcija `get_result` vraća listu srednjih vrednosti varijanse, listu standardnih devijacija, kao i krajnji histogram za sve posmatrane veličine prozora.

```
[ ]: def get_result(min_window, max_window):
    ctr, num_of_measurements = 0, 5

    # broj prozora
    num_of_cols = ((max_window - min_window) // 2) + 1

    # privremene vrednosti varijansi
    temp_vals = np.zeros((num_of_measurements, num_of_cols))

    # privremene vrednosti histograma (5x listi po jedna vrednost za svaku
    ↪ velicinu prozora u listi)
    temp_hist_vals = np.zeros((num_of_measurements, num_of_cols))
```

```

    # [[varijanse prve], [varijanse druge], [varijanse trece] , [varijanse
    ↪cetvrte], [varijanse pete]]
    for measure in range(num_of_measurements):
        # za svako merenje:

        # 1. dodaj sum na sliku lene
        lena_gauss_noise_0p01 = ski.util.random_noise(lena_img,
    ↪mode="gaussian", mean=0, var=0.01)

        # 2. [proceni varijansu za svaki prozor]
        temp_vals[ctr] = [estimate_noise_var(lena_gauss_noise_0p01, window)[0]
    ↪for window in range(min_window, max_window + 1, 2)]

        # 3. [vrednosti histograma za svaki prozor]
        temp_hist_vals[ctr] = [local_hist(lena_gauss_noise_0p01, window) for
    ↪window in range(min_window, max_window + 1, 2)]
        ctr += 1

    final_var = np.zeros(num_of_cols)
    final_stand_dev = np.zeros(num_of_cols)
    final_hist = np.zeros(num_of_cols)

    # izracunam standardne devijacije i varijanse
    for i in range(0, num_of_cols):
        final_var[i] = np.mean(temp_vals[:, i])
        final_stand_dev[i] = np.std(temp_vals[:, i])
        final_hist[i] = np.mean(temp_hist_vals[:, i])
    return final_var, final_stand_dev, final_hist

```

Nakon dobijanja rezultata pozivom funkcije `get_result`, dobijene su tri vrednosti: srednje varijanse šuma (`vars`), standardne devijacije (`standard_devs`) i usrednjene maksimalne vrednosti histograma (`hist_vals`). Veličine prozora za koje su rezultati računati definisane su nizom `windows`, koji obuhvata neparne vrednosti od minimalne veličine prozora 3 do maksimalne veličine prozora 51.

Na prvom grafiku prikazana je zavisnost procenjene varijanse šuma od veličine prozora. Tačke na grafiku predstavljaju srednje vrednosti varijanse, dok su vertikalne greške prikazane pomoću standardne devijacije. Ovakav prikaz omogućava uvid u stabilnost procene varijanse u zavisnosti od veličine prozora, kao i u rasipanje rezultata dobijenih ponovljenim merenjima.

Na drugom grafiku prikazana je zavisnost usrednjenih maksimalnih vrednosti histograma od veličine prozora. Ovaj grafikon ilustruje kako se dominantna vrednost histograma menja sa promenom veličine prozora, što daje dodatnu informaciju o ponašanju šuma i distribuciji intenziteta u signalu.

```

[ ]: vars, standard_devs, hist_vals = get_result(3, 51)
    windows = np.arange(3, 52, 2) # min_window=3, max_window=51, korak=2

```

```

# PRVI GRAFIK
plt.figure(figsize=(8,5))
plt.errorbar(windows, vars, yerr=standard_devs, fmt='-o', capsize=5,
    ↳ecolor='blue', color='red', label = "Procena šuma")
plt.xlabel("Veličina prozora")
plt.ylabel("Varijansa šuma")
plt.title("Varijansa ± standardna devijacija")
plt.legend()
plt.grid(True)
plt.show()

# DRUGI GRAFIK
plt.figure(figsize=(8,5))
plt.plot(windows, hist_vals, '-o', color='green')
plt.xlabel("Veličina prozora")
plt.ylabel("Usrednjene maks vrednosti histograma")
plt.title(r"$hist = f[window\_size]$")
plt.grid(True)
plt.show()

```

4.4 c)

Funkcija `test_automatic` za svaku zadatu vrednost varijanse izvršava pet nezavisnih merenja. Nakon sprovedenih pet merenja, na osnovu dobijenih rezultata računa se procenjena varijansa šuma, kao i odgovarajuća prosečna veličina prozora.

Greška procene definiše se kao razlika između izračunate i stvarne varijanse, odnosno:

$$\text{greška} = \text{Var}_{\text{izračunata}} - \text{Var}_{\text{prava}}$$

Na osnovu dobijenih varijansi računa se standardna devijacija.

Srednja vrednost svih veličina prozora koje su korišćene za datu varijansu se takodje racuna za merenja.

Parametar `tol` direktno utiče na izbor veličine prozora. Manje vrednosti parametra `tol` zahtevaju veće veličine prozora kako bi uslov konvergencije bio zadovoljen, dok veće vrednosti parametra `tol` dovode do izbora manjih veličina prozora.

Na kraju, funkcija `test_automatic` kao izlaz vraća srednju vrednost procenjene varijanse, standardnu devijaciju, greške i srednju veličinu prozora.

```

[ ]: def test_automatic(tol):
    # var : 0.001 -> 0.1
    # za var = 0.001: 5x izmerim varijansu pomocu automatic window, zatim
    ↳izracunam srednju vrednost i std za var = 0.001 za odredjeni prozor
    # var = 0.1 5x izmerim varijansu pomocu automatic window, mean 5x vrednosti
    ↳ , std 5x vrednosti
    num_of_measurements = 5

```

```

windows = np.zeros(num_of_measurements)
all_windows = {}
random_values = np.linspace(0.001, 0.1, 10)
var, std_deviations, errors, temp_variances = {}, {}, {}, np.
↪zeros(num_of_measurements)
for val in random_values:
    # za svaku varijansu:
    for i in range(num_of_measurements):
        # za svako merenje:
        # 1. stavi slucajan sum na varijansu
        # 2. izracunaj varijansu, prozor koji je odredjen
        lena_rnd_img = ski.util.random_noise(lena_img, mode="gaussian",
↪mean=0, var=val)
        temp_variances[i], windows[i] = estimate_noise_var(lena_rnd_img,
↪0, tol = tol)

        # 3. var[0.001] : srednja_vrednost_5xvarijansi
        # 4. greska[0.001] = dobijena - prava
        # 5. standardne devijacije
        # 6. izracunam prosečnu veličinu prozora iz 5x merenja za jednu
↪varijansu

        var[val] = np.mean(temp_variances)
        errors[val] = abs(var[val] - val)
        std_deviations[val] = np.std(temp_variances)
        all_windows[val] = np.mean(windows)
    return var, std_deviations, errors, all_windows

```

Nakon poziva funkcije `test_automatic`, dobijene su greške procene varijanse (`errs`) i veličine prozora korišćenih za svaku pravu vrednost varijanse.

Prvi grafikon prikazuje zavisnost greške procene od prave varijanse šuma, što omogućava vizuelni uvid u preciznost procene varijanse za različite nivoe šuma.

Drugi grafikon prikazuje prosečnu veličinu lokalnog susedstva (prozora) u zavisnosti od prave varijanse šuma, prikazujući kako parametar `tol` i promena varijanse utiču na izbor optimalne veličine prozora.

```

[ ]: variances, std_devs, errs, all_windows = test_automatic(0.001)

# Dijagram greške procene vs prava varijansa šuma

plt.figure(figsize=(8,5))
plt.plot(list(errs.keys()), list(errs.values()), '-o', color='red')
plt.xlabel("Prava varijansa šuma")
plt.ylabel("Greška procene")

```

```
plt.title("Greška procene varijanse šuma")
plt.grid(True)
plt.show()

# Dijagram prosečne veličine lokalnog susedstva vs prava varijansa šuma
# Izračunamo prosečne prozore po zadatoj varijansi
avg_windows = {val: np.mean(all_windows[val]) for val in all_windows}

plt.figure(figsize=(8,5))
plt.plot(list(avg_windows.keys()), list(avg_windows.values()), '-o',
        color='blue')
plt.xlabel("Prava varijansa šuma")
plt.ylabel("Prosečna veličina prozora")
plt.title("Veličina lokalnog susedstva vs prava varijansa šuma")
plt.grid(True)
plt.show()
```

4.5 DEO 4

```
[ ]: import numpy as np
import matplotlib.pyplot as plt
import skimage as ski
import time
from skimage.metrics import peak_signal_noise_ratio as psnr
from scipy.ndimage import uniform_filter, zoom
```

4.6 a)

Funkcija `box_filter` primenjuje **kockasti filter** na ulaznu sliku `img`.

Veličina filtra određena je parametrom `R`, koji definiše dimenziju prozora u kom se računa srednja vrednost.

Filter se računa pomoću funkcije `uniform_filter`, pri čemu se koristi `mode="reflect"` kako bi se ivice slike tretirale refleksijom, kao da ima ogledala pa sa druge strane su brojevi.

Rezultat funkcije je filtrirana slika gde je svaki piksel zamenjen prosečnom vrednošću unutar prozora veličine `R`.

```
[ ]: def box_filter(img, R):
    return uniform_filter(img, size = R, mode = "reflect")
```

Ako se pogleda sema rada **guided filtara** u radu *Fast self-guided filter with decimated box filters*, može se primetiti da oba filtra imaju jedan zajednički deo, a to je **računanje koeficijenata**.

Sam postupak računanja koeficijenata prikazan je u funkciji `compute_coefficients`, dok je odgovarajuća slika prikazana nakon pokretanja celije ispod.

Ova slika ilustruje kako se koeficijenti koriste u procesu filtriranja i daje vizuelni uvid u zajedničku strukturu oba filtra.

```
[ ]: img = ski.io.imread("../sekvence/ss.png")
ski.io.imshow(img)
```

Funkcija `compute_coefficients` računa koeficijente (a) i (b) za **guided filtre**. Postupak se sastoji od nekoliko koraka:

1. Kvadrira se ulazna slika `img`.
2. Računa se lokalna srednja vrednost kvadirane slike koristeći `box_filter` (leva grana formule).
3. Računa se lokalna srednja vrednost originalne slike `img` (desna grana formule).
4. Kvadrat srednje vrednosti originalne slike se računa.
5. Varijansa unutar prozora se dobija oduzimanjem kvadrata srednje vrednosti od srednje vrednosti kvadrata:

$$\text{varWin} = E(I^2) - (E(I))^2$$

6. Koeficijent (a) se računa kao:

$$a = \frac{\text{varWin}}{\text{varWin} + \epsilon}$$

7. Koeficijent (b) se računa kao:

$$b = E(I) - a \cdot E(I)$$

8. Na kraju se za oba koeficijenta računa lokalna srednja vrednost pomoću `box_filter`, i funkcija vraća `a_mean` i `b_mean`.

Ovi koeficijenti se dalje koriste u guided filteru za linearnu kombinaciju originalne slike i filtrirane komponente.

```
[ ]: def compute_coefficients(img, mask, epsilon):
    # 1 kvadriram sliku
    img_squared = img ** 2

    # 2 usrednjim kvadratnu sliku (leva grana)
    img_sqr_mean_left = box_filter(img_squared, mask)

    # 3 usrednjim obicnu sliku ( desna grana )
    img_mean = box_filter(img, mask)

    # 4 dignem obicnu sliku usrednjenu na kvadrat
    img_sqr_mean_right = img_mean ** 2

    # 5 saberi levu stranu oduzmi desnu stranu
    var_win = img_sqr_mean_left - img_sqr_mean_right

    # 6 izracunam koeficijent a
    a = var_win / (var_win + epsilon)

    # 7 izracunam b to je ovo desno dole sto se sabira
```

```

b = img_mean - a * img_mean

# 8 izracunam srednju vrednost od a
a_mean, b_mean = box_filter(a, mask), box_filter(b, mask)

return a_mean, b_mean

```

Funkcija `self_guided_dos_filter` primenjuje **self-guided filter** na ulaznu sliku `img`. Ako je slika u boji, filtriranje se vrši po kanalu, to jest funkcija se rekurzivno poziva, a rezultati se kombinuju u izlaznu sliku.

Prvo se računaju koeficijenti `a_mean` i `b_mean` pomoću funkcije `compute_coefficients`. Zatim se množi koeficijent `a_mean` sa originalnom slikom i sabira sa `b_mean` da bi se dobila filtrirana izlazna slika.

Postupak je kao na slici koja se dobija nakon pokretanja ćelije ispod.

```

[ ]: img = ski.io.imread("../sekvence/dos_guided_filter.png")
      ski.io.imshow(img)

```

Kao što se vidi, koeficijenti `a` i `b` za **box filter** su već izračunati. Jedino što preostaje jeste da se prati prikazana šema korak po korak i na osnovu nje napiše odgovarajuća funkcija koja implementira filter. Na taj način se osigurava da implementacija odgovara teorijskom modelu i prethodno definisanim parametrima.

```

[ ]: def self_guided_dos_filter(img, R, epsilon):
      if img.ndim == 3:
          output = np.zeros_like(img)
          for c in range(img.shape[2]):
              output[:, :, c] = self_guided_dos_filter(img[:, :, c], R, epsilon)
          return output

      a_mean, b_mean = compute_coefficients(img, R, epsilon)

      # 9 a_mean x img
      img_mean_product = a_mean * img

      # 10 izlazna slika
      output_img = b_mean + img_mean_product

      return output_img

```

4.7 b)

Funkcija `fast_self_guided_dos_filter` implementira **bržu verziju self-guided filtera** koristeći decimaciju slike.

Ako je ulazna slika u boji, filtriranje se radi po kanalu, a rezultati se kombinuju u izlaznu sliku.

Prvo se računa smanjen prozor `K` na osnovu originalne veličine prozora `R` i decimacije `S`.

Zatim se ulazna slika smanjuje (`decimated_img`) kako bi se ubrzalo računanje koeficijenata.

Koeficijenti `a_mean` i `b_mean` se računaju na smanjenoj slici pomoću `compute_coefficients`.

Nakon toga se koeficijenti interpoliraju nazad na originalne dimenzije slike (`zoom`) i seku na tačne dimenzije.

Na kraju se filtrirana izlazna slika dobija linearnom kombinacijom `a_mean_up * img + b_mean_up`. Ova metoda značajno ubrzava filtriranje, a i dalje zadržava detalje i ivice slike zahvaljujući guided principu.

Cela ova metoda je napisana prateći šemu koja se dobije na slici nakon pokretanja ćelije ispod korak po korak.

```
[ ]: img = ski.io.imread("../sekvence/dos_fast_guided_filter.png")
ski.io.imshow(img)
```

```
[ ]: def fast_self_guided_dos_filter(img, R, epsilon, S):
    if img.ndim == 3:
        output = np.zeros_like(img)
        for c in range(img.shape[2]):
            output[:, :, c] = fast_self_guided_dos_filter(img[:, :, c], R,
→epsilon, S)
        return output

    # 1 izracunam K
    K = R / S

    # 2 smanjim dimenziju ulazne slike
    decimated_img = img[:, :S, :S, ...]

    # 3 izracunam koeficijente iz funkcije
    a_mean, b_mean = compute_coefficients(decimated_img, K, epsilon)

    # 4 sad vratim nazad sliku na originalne dimenzije
    a_mean_up = zoom(a_mean, zoom=S, order=1)
    b_mean_up = zoom(b_mean, zoom=S, order=1)

    # 5 isecem/popunim do originalnih dimenzija (inace dobijem python exception)
    h, w = img.shape[:2]
    a_mean_up = a_mean_up[:h, :w]
    b_mean_up = b_mean_up[:h, :w]

    #6 izracunam izlaznu sliku
    output_img = a_mean_up * img + b_mean_up

    return output_img
```

4.8 c)

```
[ ]: def plt_img(img1, img2, title2):
    fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(10,7), dpi=80)
    ax = axes.ravel()

    ax[0].imshow(img1, cmap = 'gray'); ax[0].set_title('Ulazna slika')
    ax[1].imshow(img2, cmap='gray', vmin = 0, vmax = 1); ax[1].
    ↪set_title(title2);
    plt.show()
```

Funkcija `calc_psnr` računa **PSNR (Peak Signal-to-Noise Ratio)** između slika dobijenih korišćenjem `dos_guided` i `fast_dos_guided` algoritama.

Ako je ulazna slika u boji (RGB), PSNR se računa posebno po svakom kanalu, a zatim se te tri vrednosti prosečno kombinuju da bi se dobila ukupna vrednost.

U slučaju da je slika grayscale, PSNR se računa klasičnim pozivom funkcije, direktno na celoj slici.

Ovaj postupak omogućava poređenje kvaliteta filtriranih slika u odnosu na originalnu sliku.

```
[ ]: def calc_PSNR(img, img_dos_guided, img_fast):
    # PSNR između slika
    if img.ndim == 3:
        # Za RGB, računaj po kanalima
        psnr_val = np.mean([
            psnr(img_dos_guided[:, :, c], img_fast[:, :, c], data_range=1.0)
            for c in range(3)
        ])
    else:
        psnr_val = psnr(img_dos_guided, img_fast, data_range=1.0)

    return psnr_val
```

Funkcija `test_one_image` vrši testiranje **guided** i **fast self-guided DOS** filtra na jednoj slici `img`.

Prvo se meri vreme filtriranja običnim `self_guided_dos_filter` i čuva se kao `t_img_guided`. Zatim se meri vreme filtriranja brzim `fast_self_guided_dos_filter` algoritmom i čuva se kao `t_img_fast`.

Na kraju se računa PSNR između rezultata oba filtra pomoću funkcije `calc_PSNR`.

Funkcija vraća vreme izvršavanja oba filtra i vrednost PSNR, što omogućava poređenje kvaliteta i brzine oba algoritma.

```
[ ]: def test_one_image(img, R_val, eps, s = 2):
    start = time.time()
    img_dos_guided = self_guided_dos_filter(img, R_val, eps)
    end = time.time()
    t_img_guided = end - start
```

```

# Brzi filter
start = time.time()
img_fast = fast_self_guided_dos_filter(img, R_val, eps, 2)
end = time.time()
t_img_fast = end - start

PSNR_VAL = calc_PSNR(img, img_dos_guided, img_fast)

return t_img_guided, t_img_fast, PSNR_VAL

```

Sledeća ćelija prikazuje testiranje implementacije oba filtra i demonstrira ispravnost funkcija.

Proces testiranja se vrši na dve slike: jedna je **grayscale**(lena), dok je druga **slika u boji** (RGB papagaji).

Prikazani su rezultati primene oba filtra na obe slike nakon izvršenja ćelije ispod.

```

[ ]: lena_tif = ski.util.img_as_float(ski.io.imread(data_dir + "lena.tif"))
lena_guide = self_guided_dos_filter(lena_tif, 9, 0.1)
plt_img(lena_tif, lena_guide, "Self guided filtrirana slika")

lena_guide_fast = fast_self_guided_dos_filter(lena_tif, 15, 0.1, 2)
plt_img(lena_tif, lena_guide_fast, "Fast Self guided filtrirana slika")

parrots = ski.util.img_as_float(ski.io.imread(data_dir + "papagaji.jpg"))
parrots_guide = self_guided_dos_filter(parrots, 9, 0.1)
plt_img(parrots, parrots_guide, "Self guided filtrirana slika")

parrots_guide_fast = fast_self_guided_dos_filter(parrots, 15, 0.1, 2)
plt_img(parrots, parrots_guide_fast, "Fast Self guided filtrirana slika")

```

```

[ ]: data_dir = "../sekvence/"
lena_tif = ski.util.img_as_float(ski.io.imread(data_dir + "lena.tif"))
parrots = ski.util.img_as_float(ski.io.imread(data_dir + "papagaji.jpg"))

R_values = [3, 5, 7, 9, 11, 13, 15, 17, 19, 21, 25, 31, 39, 47, 55, 63] # 16
    ↪ vrednosti
epsilon_values = [0.0001, 0.001, 0.01, 0.1, 1.0]

# Strukture za čuvanje rezultata
PSNR_values_lena = {} # ključ: (R, epsilon), vrednost: PSNR
PSNR_values_parrots = {}

t_lena_guided, t_parrots_guided = {}, {}
t_lena_fast, t_parrots_fast = {}, {}

```

Ova ćelija vrši **testiranje oba filtra** za sve kombinacije veličine prozora R i parametra ϵ . Za svaku kombinaciju parametara se poziva funkcija `test_one_image` na slikama **Lena** i **Papagaji**.

Vreme izvršavanja i PSNR vrednosti se čuvaju u odgovarajućim rečnicima (`t_lena_guided`, `t_lena_fast`, `PSNR_values_lena`,...), posebno za svaku sliku.

Ovo omogućava detaljno poređenje kvaliteta filtriranja i performansi oba filtra za sve parametre.

```
[ ]: # Testiram za sve kombinacije R i epsilon
for R_val in R_values:
    for eps in epsilon_values:
        # LENA
        t_guided, t_fast, psnr_val = test_one_image(lena_tif, R_val, eps, 2)
        t_lena_guided[(R_val, eps)] = t_guided
        t_lena_fast[(R_val, eps)] = t_fast
        PSNR_values_lena[(R_val, eps)] = psnr_val

        # PARROTS
        t_guided, t_fast, psnr_val = test_one_image(parrots, R_val, eps, 2)
        t_parrots_guided[(R_val, eps)] = t_guided
        t_parrots_fast[(R_val, eps)] = t_fast
        PSNR_values_parrots[(R_val, eps)] = psnr_val
```

Ova ćelija prikazuje **PSNR vrednosti** za slike **Lena** i **Papagaji** za sve kombinacije veličine prozora R i parametra ϵ .

Za svaku kombinaciju parametara se štampa PSNR u decibelima.

Rezultati pomažu da se vidi kako različite veličine prozora i vrednosti epsilon utiču na očuvanje detalja i smanjenje šuma.

Na taj način mogu lako identifikovati optimalne parametre za oba filtra.

```
[ ]: print("\nLENA:")
for (R, eps), psnr_val in PSNR_values_lena.items():
    print(f"R={R}, = {eps}: PSNR={psnr_val:.1f}dB")

print("\nPARROTS:")
for (R, eps), psnr_val in PSNR_values_parrots.items():
    print(f"R={R}, = {eps}: PSNR={psnr_val:.1f}dB")
```

Ova ćelija izračunava **prosečna vremena izvršavanja** oba filtra za svaku veličinu prozora R .

Za svaku vrednost R se izvlače sva merenja po svim vrednostima ϵ za slike **Lena** i **Papagaji**.

Zatim se računa prosečno vreme za obični i brzi filter, koje se čuva u listama `avg_time_guided_lena`, `avg_time_fast_lena`, `avg_time_guided_parrots` i `avg_time_fast_parrots`.

Ovi proseci se kasnije koriste za crtanje dijagrama brzine rada i poređenje performansi oba filtra.

```
[ ]: avg_time_guided_lena = []
avg_time_fast_lena = []
avg_time_guided_parrots = []
avg_time_fast_parrots = []
```

```

for R in R_values:
    # Izvuci svo vremena za dati R (za sve )
    times_guided_lena = [t_lena_guided[(R, eps)] for eps in epsilon_values]
    times_fast_lena = [t_lena_fast[(R, eps)] for eps in epsilon_values]

    times_guided_parrots = [t_parrots_guided[(R, eps)] for eps in
↳epsilon_values]
    times_fast_parrots = [t_parrots_fast[(R, eps)] for eps in epsilon_values]

    # Izračunam proseke: [R = 3 prosek, R = 7 prosek, R = 15 prosek ,...]
    avg_time_guided_lena.append(np.mean(times_guided_lena))
    avg_time_fast_lena.append(np.mean(times_fast_lena))
    avg_time_guided_parrots.append(np.mean(times_guided_parrots))
    avg_time_fast_parrots.append(np.mean(times_fast_parrots))

```

Prvi dijagram prikazuje **vreme izvršavanja** običnog i brzog self-guided filtera za sliku **Lena** u zavisnosti od veličine prozora R.

Drugi dijagram prikazuje istu analizu za sliku **Papagaji** (boja).

Na oba dijagrama se jasno vidi kako brzina rada raste sa veličinom prozora, a brzi filter (S=2) značajno smanjuje vreme izvršavanja u odnosu na obični filter.

Legenda razlikuje rezultate običnog filtera i brze varijante, dok grid i oznake olakšavaju vizuelnu interpretaciju.

```

[ ]: # DIJAGRAM 1: LENA
plt.figure(figsize=(10, 6))
plt.plot(R_values, avg_time_guided_lena, 'b-o', linewidth=2, markersize=8,
↳label='Obični filter')
plt.plot(R_values, avg_time_fast_lena, 'r-s', linewidth=2, markersize=8,
↳label='Brzi filter (S=2)')
plt.xlabel('Veličina prozora R', fontsize=12)
plt.ylabel('Vreme izvršavanja (s)', fontsize=12)
plt.title('Brzina rada - Lena', fontsize=14, fontweight='bold')
plt.legend()
plt.grid(True, alpha=0.3)
plt.xticks(R_values)
plt.show()

```

```

[ ]: # DIJAGRAM 2: PAPAGAJI (ZASEBNI)
plt.figure(figsize=(10, 6))
plt.plot(R_values, avg_time_guided_parrots, 'b-o', linewidth=2, markersize=8,
↳label='Obični filter')
plt.plot(R_values, avg_time_fast_parrots, 'r-s', linewidth=2, markersize=8,
↳label='Brzi filter (S=2)')
plt.xlabel('Veličina prozora R', fontsize=12)
plt.ylabel('Vreme izvršavanja (s)', fontsize=12)

```

```
plt.title('Brzina rada - Papagaji', fontsize=14, fontweight='bold')
plt.legend()
plt.grid(True, alpha=0.3)
plt.xticks(R_values)
plt.show()
```

4.9 Bonus

```
[ ]: import numpy as np
import skimage as ski
import matplotlib.pyplot as plt
```

Funkcija `create_pyramid` kreira **piramidu slika** gde je svaka naredna slika dobijena **smanjenjem rezolucije za faktor 2** u odnosu na prethodnu.

Proces se odvija tako što se prvo odabira svaka druga vrednost po visini i širini, čime se rezolucija prepolovi.

Sve generisane slike se čuvaju u listi, a na kraju se redosled liste invertuje tako da: - prva slika u listi ima **najmanju rezoluciju** (najgrublji nivo), - poslednja slika ima **originalnu rezoluciju**.

Ovakav raspored omogućava obradu slike po principu *coarse-to-fine*, što je posebno pogodno za procenu kernela i iterativno uklanjanje zamućenja.

```
[ ]: def create_pyramid(img, num_scales):
    pyr = [img]
    for _ in range(1, num_scales):
        img = ski.filters.gaussian(img, sigma=1)
        img = img[::2, ::2]
        pyr.append(img)
    return pyr[::-1] # coarse to fine
```

```
[ ]: def init_LK(pyramid, ksize):
    L = pyramid[0].copy()
    K = np.zeros((ksize, ksize))
    K[ksize//2, ksize//2] = 1.0
    return L, K
```

4.9.1 1. Bilateralni filter (bilateral)

Ova funkcija primenjuje bilateralni filter na ulaznu sliku. Cilj bilateralnog filtriranja je uklanjanje šuma uz istovremeno očuvanje ivica. Za razliku od klasičnih linearnih filtara, bilateralni filter uzima u obzir i prostornu udaljenost piksela i razliku u intenzitetu, zbog čega se ivice ne zamućuju. Koristi se kao metoda za edge-preserving denoising, naročito kada je važno zadržati strukturalne detalje slike.

4.9.2 2. Shock filter (shock)

Ova funkcija implementira nelinearni shock filter koji služi za izoštravanje ivica na slici. Zasniva se na kombinaciji Laplasovog operatora i magnitude gradijenta. Laplasijan određuje da li se piksel nalazi u svijetloj ili tamnoj regiji u odnosu na okolinu, dok gradijent daje informaciju o jačini ivice. Rezultat je pojačavanje diskontinuiteta i stvaranje oštrijih, jasnije definisanih ivica, uz kontrolu jačine efekta pomoću vremenskog koraka.

4.9.3 3. Pragovanje gradijenta (gradient_threshold)

Ova funkcija izdvaja najznačajnije ivice na slici tako što računa gradijent i zadržava samo one delove sa najvećom magnitudom. Nakon izračunavanja jačine gradijenta, određuje se prag na osnovu zadatog procenta koji definiše koliko najjačih ivica treba sačuvati. Slabiji gradijenti se uklanjaju, čime se dobija redak prikaz ivica. Ovaj pristup je koristan u detekciji ivica, redukciji šuma i u variacionim metodama obrade slike.

```
[ ]: def bilateral(L):  
    return ski.restoration.denoise_bilateral(L, sigma_spatial=2, sigma_color=0.  
    ↪05)
```

```
[ ]: def shock(L, dt):  
    lap = ski.filters.laplace(L)  
    grad = np.sqrt(np.square(ski.filters.sobel_h(L)) + np.square(ski.filters.  
    ↪sobel_v(L)))  
    return L - dt * np.sign(lap) * grad
```

```
[ ]: def gradient_threshold(L, thresh):  
    """Threshold gradient adaptively, decaying thresh per iteration."""  
    Px = ski.filters.sobel_h(L)  
    Py = ski.filters.sobel_v(L)  
    mag = np.sqrt(Px**2 + Py**2)  
    thresh_val = np.percentile(mag, (1 - thresh) * 100) # keep top fraction  
    mask = mag > thresh_val  
    return Px * mask, Py * mask
```

```
[ ]: def normalize_kernel(K):  
    K[K < K/20] = 0  
    s = K.sum()  
    if s > 0:  
        K /= s  
    return K
```

4.9.4 Procena kernela zamućenja (estimate_kernel)

Ova funkcija služi za procenu kernela zamućenja (blur kernel / PSF) na osnovu gradijenata oštre slike i zamućene slike. .

Na početku se iz zamućene slike izdvajaju samo najjači gradijenti kako bi se smanjio uticaj šuma i slabih ivica. Zatim se procena kernela vrši u **Fourierovom domenu**, gde se problem formuliše kao regularizovani least-squares odnos između gradijenata oštne i zamućene slike. Parametar regularizacije sprečava numeričku nestabilnost i preuveličavanje šuma.

Dobijeni kernel se vraća u prostorni domen, centrira i zatim se iseca na željenu veličinu. Nakon toga se primenjuje dodatna stabilizacija u vidu Gaussovog prigušenja ka ivicama kernela, čime je cilj da se izvuku kompaktni i glatki kernel oblici. Na kraju se kernel normalizuje tako da ukupna suma ostane kontrolisana.

Rezultat funkcije je stabilan i fizički smislen kernel zamućenja koji najbolje objašnjava odnos između ulaznih gradijenata slike i zamućenog posmatranja.

```
[ ]: def estimate_kernel(Px, Py, B, ksize, beta):
    Bx, By = gradient_threshold(B, 0.1)

    nominator = 0
    denominator = beta

    for P, Bgrad in [(Px, Bx), (Py, By)]:
        Pf = np.fft.fft2(P)
        Bf = np.fft.fft2(Bgrad)
        nominator += np.conj(Pf) * Bf
        denominator += np.conj(Pf) * Pf

    K = np.real(np.fft.ifft2(nominator / (denominator + 1e-8)))
    K = np.fft.fftshift(K)

    # Crop to kernel size
    c = np.array(K.shape) // 2
    r = ksize // 2
    K = K[c[0]-r:c[0]+r+1, c[1]-r:c[1]+r+1]

    # Stabilizacija
    sigma = ksize / 4
    yy, xx = np.indices(K.shape)
    cy, cx = K.shape[0] // 2, K.shape[1] // 2
    K *= np.exp(-((yy - cy) ** 2 + (xx - cx) ** 2) / (2 * sigma ** 2))
    K = normalize_kernel(K)

    return K
```

4.9.5 Konverzija PSF u OTF (psf2otf)

Ova funkcija pretvara **PSF (Point Spread Function)** iz prostornog domena u **OTF (Optical Transfer Function)** Fourier domenu. OTF predstavlja frekvencijski odziv sistema i koristi se u dekonvoluciji i filtriranju slike u frekvencijskom domenu.

Funkcija najpre smešta PSF u matricu zadate veličine, popunjavajući ostatak nulama (zero-

padding), kako bi dimenzije odgovarale slici nad kojom se radi. Zatim se kernel centrira, što je neophodno za ispravnu Fourierovu transformaciju.

Na kraju se primenjuje dvodimenzionalna Fourierova transformacija, čime se dobija OTF. Rezultat se koristi za efikasne konvolucije i dekonvolucije u frekvencijskom domenu.

```
[ ]: def psf2otf(psf, shape):
    pad = np.zeros(shape)
    h, w = psf.shape
    pad[:h, :w] = psf
    pad = np.roll(pad, -h//2, 0)
    pad = np.roll(pad, -w//2, 1)
    return np.fft.fft2(pad)
```

4.9.6 Dekonvolucija slike (deconvolve)

Ova funkcija vrši **regularizovanu dekonvoluciju** zamućene slike koristeći poznat kernel zamućenja. Cilj je rekonstrukcija latentne (oštre) slike iz zamućenog posmatranja, uz kontrolu šuma i numeričke nestabilnosti.

Proces se odvija u **Fourierovom domenu**, gde se konvolucija slike i kernela zamućenja može efikasno predstaviti kao množenje u frekvencijskom prostoru. Kernel se najpre transformiše u OTF, dok se zamućena slika prevodi u frekvencijski domen pomoću Fourierove transformacije. Zatim se uvodi skup diskretnih diferencijalnih operatora koji aproksimiraju izvode nultog, prvog i drugog reda, čime se obezbeđuje regularizacija rešenja i smanjenje uticaja šuma.

Svaki diferencijalni operator doprinosi numeričkom i nazivničkom delu rešenja u skladu sa dodeljenom težinom. Time se dobija balans između vernosti podacima i glatkoće rešenja. Parametar regularizacije kontroliše koliko će se potisnuti šum i nestabilne visoke frekvencije.

Na kraju se rezultat vraća u prostorni domen, uzima se realni deo, i vrednosti se ograničavaju na dozvoljeni opseg intenziteta. Rezultat je procena originalne slike.

```
[ ]: def deconvolve(B, K, alpha):
    h, w = B.shape

    # 1. Kernel OTF
    Kf = psf2otf(K, (h, w))

    # 2. FFT slike (bez shift-a!)
    Bf = np.fft.fft2(B)

    derivs = {
        "0": np.ones((1, 1)),
        "x": np.array([[ -1, 1]]),
        "y": np.array([[ -1], [ 1]]),
        "xx": np.array([[ 1, -2, 1]]),
        "yy": np.array([[ 1], [-2], [ 1]]),
        "xy": np.array([[ 1, -1], [-1, 1]])
    }
```

```

weights = {"0": 1, "x": 1, "y": 1, "xx": 0.5, "yy": 0.5, "xy": 0.5}

num = 0
den = alpha

for k, v in derivs.items():
    D = np.zeros((h, w))
    v = np.array(v)
    D[:v.shape[0], :v.shape[1]] = v
    D = np.roll(D, -v.shape[0] // 2, 0)
    D = np.roll(D, -v.shape[1] // 2, 1)
    Df = np.fft.fft2(D)
    num += weights[k] * np.conj(Kf * Df) * (Df * Bf)
    den += weights[k] * np.abs(Kf * Df) ** 2

Lf = num / (den + 1e-8)
L = np.real(np.fft.ifft2(Lf))
L = np.clip(L, 0, 1)

return L

```

Sam postupak algoritma se sastoji od sledećih koraka:

1. Izgradnja piramide slika

Napravimo piramidu slika, gde se na dnu nalazi slika sa najmanjom rezolucijom, dok je na vrhu slika sa originalnom rezolucijom. Svaka slika u piramidi predstavlja jedan nivo.

2. Određivanje broja iteracija po nivou

Za svaki nivo piramide definiše se broj iteracija koje će algoritam izvršiti na tom nivou.

3. Iteracije po nivou

Za svaki nivo piramide vrši se unapred definisan broj iteracija. Svaka iteracija se sastoji iz sledećih koraka:

3.1 Prediction korak

U ovom koraku primenjuju se tri metode nad trenutnom slikom:

- Bilateralni filter – za uklanjanje šuma uz očuvanje ivica.
- Shock filter – za pojačavanje i izoštravanje ivica.
- Gradient threshold – za selekciju najjačih gradijenata (ivica).

3.2 Procena kernela zamućenja

Na osnovu trenutne slike i rezultata iz predikcionog koraka, algoritam pokušava da odredi kernel zamućenja (Point Spread Function, PSF) koji najbolje opisuje način na koji je slika zamućena.

3.3 Dekonvolucija

Koristeći procenjeni kernel, algoritam pokušava da rekonstruiše originalnu sliku iz zamućene,

što predstavlja klasičan problem **blind deconvolution**, s obzirom da je slika data formulom:

$$B = L * K + N$$

gde je B zamućena slika, L originalna slika, K kernel zamućenja, a N šum.

4. Prolazak kroz sve nivoe piramide

Nakon završetka iteracija na jednom nivou, rekonstruisana slika se **upsample-uje** i koristi kao inicijalna slika za sledeći nivo piramide.

Nakon prolaska kroz sve nivoe piramide, dobija se finalna slika koja je odzamućena korišćenjem ovog algoritma.

```
[ ]: def process_one_pyramid_level(L, pyramid_img, num_of_iters, ksize, beta, alpha,
    ↪dt, sigma_r):
    thresh = 0.1 # initial gradient threshold fraction
    for _ in range(num_of_iters):
        Lb = bilateral(L)
        Ls = shock(Lb, dt)
        Px, Py = gradient_threshold(Ls, thresh)

        K = estimate_kernel(Px, Py, pyramid_img, ksize, beta)
        L = deconvolve(pyramid_img, K, alpha)

        # decay parameters
        dt *= 0.9
        thresh = max(thresh * 0.9, 0.03) # lagano decaying, ne previše

    return L, K
```

```
[ ]: def blind_deblur_gray(B, ksize, beta, alpha):
    pyramid = create_pyramid(B, 5)
    L, K = init_LK(pyramid, ksize)

    sigma_r = 0.5
    dt = 1

    for i in range(len(pyramid)):
        L, K = process_one_pyramid_level(L, pyramid[i], 10, ksize, beta, alpha,
    ↪dt, sigma_r)

        if i < len(pyramid)-1:
            L = ski.transform.resize(L, pyramid[i+1].shape, anti_aliasing=True)
            K[K < 0] = 0
            K /= K.sum() + 1e-8

    return L, K
```

Funkcija **blind_deblur_gray** primenjuje opisani algoritam na **grayscale slike**.

Funkcija `fast_motion_deblur_rgb` vrši isti algoritam na **RGB slikama**, ali ne obrađuje svaki kanal posebno, da bi se izbeglo višestruko izvođenje teških proračuna. Umesto toga, RGB slika se prvo konvertuje u **YCbCr** prostor boja, zatim se izračuna **luminansa (Y kanal)** i algoritam se primenjuje samo na njemu.

Nakon što je luminansa odzamućena, rezultat se kombinuje sa originalnim **Cb i Cr kanalima**, i slika se vraća u **RGB domen**. Na taj način se dobija odzamućena RGB slika, a kernel zamućenja se može koristiti za dalju obradu ili vizuelizaciju.

```
[ ]: def fast_motion_deblur_rgb(img, ksize, beta, alpha):
    img = ski.util.img_as_float(img)
    ybcr = ski.color.rgb2ybcr(img)
    Y = ybcr[..., 0] / 255.0
    restored_Y, K = blind_deblur_gray(Y, ksize, beta, alpha)
    restored_Y = np.clip(restored_Y, 0, 1)
    ybcr[..., 0] = restored_Y * 255.0
    restored_rgb = ski.color.ybcr2rgb(ybcr)
    restored_rgb = np.clip(restored_rgb, 0, 1)
    return restored_rgb, K
```

4.9.7 Funkcija `dos_fast_motion_deblurring`

Ova funkcija predstavlja **glavni interfejs** za odzamućivanje slike. Na osnovu dimenzija ulazne slike automatski odlučuje koji algoritam će se primeniti:

- Ako je ulazna slika **RGB** (`ndim == 3`), poziva se funkcija `fast_motion_deblur_rgb`, koja primenjuje algoritam nad **luminansom Y-kanala** u YCbCr prostoru i vraća rekonstruisanu RGB sliku.
- Ako je ulazna slika **grayscale** (`ndim == 2`), koristi se funkcija `blind_deblur_gray`, koja radi isti algoritam direktno nad jednim kanalom.
- Ako slika nije ni RGB ni grayscale, funkcija baca grešku (`TypeError`) i upozorava korisnika da ulaz nije validan.

Funkcija vraća **odzamućenu sliku** istih dimenzija kao ulazna, spremnu za dalju obradu ili vizuelizaciju.

```
[ ]: def dos_fast_motion_deblurring(img, ksize=15, beta=5, alpha=0.1):
    if img.ndim == 3:
        output_img, _ = fast_motion_deblur_rgb(img, ksize, beta, alpha)
    elif img.ndim == 2:
        output_img, _ = blind_deblur_gray(img, ksize, beta, alpha)
    else:
        raise TypeError("Error! img must be rgb or grayscale")
    return output_img
```

Na slici ispod prikazana je **originalna slika** sa leve strane, dok se sa desne strane vidi **odzamućena slika** dobijena primenom algoritma. Može se uočiti da algoritam funkcioniše za zadate parametre, iako rezultat nije potpuno idealan, posebno u oblastima sa složenim detaljima ili jačim zamućenjima.

```
[ ]: data_dir = "../sekvence/"
img = ski.util.img_as_float(ski.io.imread(data_dir + "blurry_dolls.png"))
img = img[:, :, :3]

result_img = dos_fast_motion_deblurring(img, ksize=11, beta=5, alpha=0.1)

plt.figure(figsize=(14,5))
plt.subplot(1,2,1); plt.title("Blurred"); plt.imshow(img); plt.axis("off")
plt.subplot(1,2,2); plt.title("Restored"); plt.imshow(result_img); plt.
    ↪axis("off")
plt.show()
```

Analiza grafika u **frekvencijskom domenu** pokazuje da horizontalne linije na slici generišu **vertikalne linije** u spektru, dok vertikalne linije na slici proizvode **horizontalne linije** u spektru.

Ukoliko bi bilo potrebno ukloniti ovakve smetnje, mogla bi se primeniti **notch filtracija**. Na primer, filtriranjem horizontalnih linija u spektru, vertikalne linije bi nestale sa same slike. Analogno, filtriranjem vertikalnih linija u spektru, uklonile bi se horizontalne linije sa slike. Ovaj princip omogućava selektivno uklanjanje periodičnog šuma ili neželjenih obrazaca u slici.

```
[ ]: F = np.fft.fft2(result_img[..., 0] if result_img.ndim == 3 else result_img)
F_shift = np.fft.fftshift(F)
mag = np.log(1 + np.abs(F_shift))
ski.io.imshow(mag, cmap = 'gray')
```